# PROPOSAL PROJECT

## MATA KULIAH KECERDASAN BUATAN

**JUDUL PROPOSAL**

# PHISHGUARD: APLIKASI DETEKSI SMS/EMAIL PHISHING


**PENGUSUL**

[NIM] [Nama Mahasiswa] (Ketua)
[NIM] [Nama Mahasiswa] (Anggota 1)
[NIM] [Nama Mahasiswa] (Anggota 2)
[NIM] [Nama Mahasiswa] (Anggota 3)

**PROGRAM STUDI INFORMATIKA (S1)**

**FAKULTAS TEKNIK DAN INFORMATIKA**

**UNIVERSITAS BINA SARANA INFORMATIKA**


**2026**

## IDENTITAS PROJECT DAN PETUNJUK PENGGUNAAN

| | |
|---|---|
| **Mata Kuliah** | Kecerdasan Buatan |
| **Judul Project** | PhishGuard: Aplikasi Deteksi SMS/Email Phishing |
| **Kelompok/Kelas** | [Kelompok] / [Kode kelas] |
| **Studi Kasus** | Pengguna ponsel sulit membedakan SMS/WhatsApp/email penipuan (phishing) dari pesan resmi sebelum mengklik tautan, memasang file APK, atau menyebutkan kode OTP |
| **Platform** | Flutter: Android (APK) dan web |
| **Backend/API** | FastAPI (Python 3.14), dikemas Docker |
| **Model AI** | Pipeline TF-IDF n-gram kata dan karakter + Logistic Regression (scikit-learn), dilengkapi lapisan aturan |
| **Repository** | https://github.com/lildwagz/phishguard |
| **Live App** | [URL aplikasi, diisi pada laporan akhir] |

## LEMBAR PERSETUJUAN

Proposal project mata kuliah Kecerdasan Buatan dengan judul "PhishGuard: Aplikasi Deteksi SMS/Email Phishing" telah diperiksa dan disetujui untuk dilaksanakan.

| | |
|---|---|
| **Kelompok** | [Nomor/Nama Kelompok] |
| **Kelas** | [Kode Kelas] |
| **Semester** | [Semester/Tahun Akademik] |
| **Tanggal** | [Tanggal Bulan Tahun] |

| **Ketua Kelompok** | **Dosen Pengampu** |
|---|---|
| [Nama Mahasiswa] | [Nama Dosen] |
| NIM: [NIM] | NIDN: [NIDN] |

# DAFTAR ISI

LEMBAR PERSETUJUAN/PENGESAHAN

RINGKASAN PROPOSAL

BAB I PENDAHULUAN

BAB II ANALISIS KEBUTUHAN DAN LANDASAN SOLUSI

BAB III RANCANGAN ARSITEKTUR DAN API

BAB IV RENCANA IMPLEMENTASI DAN PENGUJIAN

BAB V MANAJEMEN PROJECT

DAFTAR PUSTAKA

LAMPIRAN

## RINGKASAN PROPOSAL

Penipuan lewat pesan teks masih banyak menimpa masyarakat Indonesia. Indonesia Anti-Scam Centre (IASC) yang dikelola OJK menerima 411.055 laporan penipuan keuangan sejak November 2024 sampai 23 Desember 2025, dengan total kerugian yang dilaporkan sekitar Rp9 triliun (Al Hakim, 2026). Banyak penipuan dimulai dari SMS atau pesan WhatsApp yang meniru bank, kurir, marketplace, atau instansi pemerintah, lalu meminta korban mengklik tautan, memasang file APK, atau menyebutkan kode OTP.

PhishGuard ditujukan untuk pengguna ponsel umum yang menerima pesan mencurigakan dan ingin memeriksanya sebelum bertindak. Pengguna menempel teks pesan ke aplikasi Flutter, memilih teks di aplikasi lain lalu menekan menu "Scan dengan PhishGuard", atau membagikan SMS ke PhishGuard lewat menu Bagikan. Aplikasi mengirim teks ke REST API FastAPI, dan API mengembalikan label Phishing atau Aman, skor keyakinan (confidence), serta alasan bila ada tanda bahaya yang dikenali.

Model AI-nya berupa pipeline TF-IDF (n-gram kata dan karakter) dengan Logistic Regression yang dilatih pada 5.654 pesan berbahasa Inggris dan Indonesia dari dataset publik (Mishra & Soni, 2022; Rahmi & Wibisono, 2016; bopbi, 2021). Model dilengkapi lapisan aturan untuk pola yang jarang muncul di data latih, misalnya lampiran .apk dan permintaan kode OTP. Pada 1.413 pesan uji, prototype mencapai akurasi 97,8% dan F1 kelas phishing 0,921. Latency HTTP p95 4,5 ms pada 300 request di Docker lokal. Backend dikemas dengan Docker, dan aplikasi sudah berjalan sebagai APK Android dan aplikasi web.

Luaran project adalah prototype aplikasi, repository kode, dokumentasi API, dan laporan. Manfaatnya bagi pengguna adalah pemeriksaan cepat sebelum mengklik tautan atau mentransfer uang. Hasil aplikasi tetap berupa bantuan awal, dan keputusan akhir ada pada pengguna.

| | |
|---|---|
| **Kata Kunci** | phishing; klasifikasi teks; TF-IDF; logistic regression; Flutter |
| **Luaran Utama** | Prototype aplikasi Flutter (APK Android dan web), REST API FastAPI dalam container Docker, repository kode, dokumentasi API (/docs), dan laporan |

# BAB I PENDAHULUAN

## 1.1 Latar Belakang

OJK melalui Indonesia Anti-Scam Centre (IASC) mencatat 411.055 laporan penipuan keuangan sejak IASC diluncurkan pada November 2024 sampai 23 Desember 2025. Total kerugian dana yang dilaporkan sebesar Rp9 triliun, dan 681.890 rekening dilaporkan terkait penipuan tersebut (Al Hakim, 2026).

Pesan teks adalah salah satu pintu masuk penipuan. Berita cek fakta dan peringatan resmi bank sepanjang 2022 sampai 2026 memuat contoh pesan yang meniru perubahan tarif transfer bank, paket kurir yang tertahan, surat tilang elektronik, undangan pernikahan berupa file APK, bantuan sosial, dan lowongan kerja paruh waktu (contohnya dalam Tim Cek Fakta, 2022, 2023; Bank Central Asia, 2025). Pesan seperti ini meniru pengirim resmi, sehingga sulit dibedakan dari notifikasi asli dalam sekali baca, apalagi oleh pengguna yang sedang terburu-buru.

Saat ini pengguna biasanya mengandalkan pengalaman sendiri, peringatan dari bank, atau berita cek fakta yang terbit setelah suatu modus menyebar. Pemeriksaan manual seperti itu lambat dan tidak selalu tersedia ketika pesan diterima. Klasifikasi teks dengan machine learning dapat mempelajari pola kata dari ribuan contoh pesan berlabel dan memberi penilaian dalam hitungan milidetik. Karena modus penipuan berubah, model perlu dilatih ulang ketika data baru tersedia, dan pola yang belum ada di data latih ditangani dengan aturan tambahan.

## 1.2 Identifikasi dan Rumusan Masalah

Persoalan computing utama adalah klasifikasi teks biner: menentukan apakah sebuah pesan SMS/email termasuk phishing atau aman, beserta tingkat keyakinannya, dan menyajikannya sebagai layanan yang dapat dipanggil oleh aplikasi mobile.

Akar masalah yang diidentifikasi:

- Pesan penipuan meniru gaya bahasa pengirim resmi, sehingga sulit dibedakan secara manual.
- Data berlabel untuk SMS penipuan berbahasa Indonesia sedikit dan sebagian besar berasal dari modus lama (undian berhadiah). Modus baru seperti file APK dan permintaan OTP hampir tidak ada di dataset publik.
- Jumlah pesan phishing jauh lebih sedikit daripada pesan aman (sekitar 14% pada data yang digunakan), sehingga akurasi saja tidak cukup untuk menilai model.

Rumusan masalah project:

- Bagaimana membangun model klasifikasi teks yang membedakan pesan phishing dan aman dalam bahasa Indonesia dan Inggris dengan akurasi minimal 85% dan F1 kelas phishing minimal 0,85 pada data uji?
- Bagaimana merancang REST API yang memvalidasi input, menjalankan inference, dan mengembalikan label, confidence, serta alasan dalam waktu kurang dari 3 detik?
- Bagaimana menyajikan hasil tersebut di aplikasi Flutter agar pengguna dapat memeriksa pesan dari aplikasi PhishGuard maupun langsung dari aplikasi pesan lain?

### 1.3 Tujuan Project

- Menghasilkan model klasifikasi teks dengan akurasi minimal 85% dan F1 kelas phishing minimal 0,85 pada data uji yang dipisahkan dari data latih.
- Menyediakan REST API dengan endpoint GET /health dan POST /v1/predict yang menolak input kosong atau lebih dari 2.000 karakter dengan status 422, membatasi penyalahgunaan dengan rate limiting dan batas ukuran body, dan menjawab prediksi dengan latency p95 di bawah 3 detik.
- Menyediakan aplikasi Flutter (Android dan web) yang menerima teks, menampilkan label, confidence, dan alasan, menyimpan riwayat di perangkat, serta dapat dipanggil dari menu seleksi teks dan menu Bagikan Android.

- Menyediakan layanan yang dapat dijalankan ulang dengan satu perintah Docker Compose, dan menyiapkan rencana deployment ke cloud.

### 1.4 Manfaat Project

Bagi pengguna, PhishGuard memberi penilaian awal sebelum mereka mengklik tautan, memasang aplikasi, menyebutkan kode OTP, atau mentransfer uang. Alasan yang ditampilkan (misalnya "Meminta membuka/menginstal file .apk") juga membantu pengguna mengenali pola penipuan.

Bagi organisasi seperti bank atau penyedia layanan pesan, rancangan API yang terpisah dari tampilan memungkinkan layanan yang sama dipakai oleh klien lain, misalnya halaman web atau chatbot layanan pelanggan.

Bagi mahasiswa, project ini melatih seluruh tahap rekayasa aplikasi AI sesuai materi mata kuliah: analisis kebutuhan, perancangan API, serialization model, frontend Flutter, containerization, pengujian, dan monitoring.

Bagi pengembangan aplikasi AI, project ini menunjukkan cara menggabungkan model statistik dengan aturan eksplisit ketika data latih belum mencakup pola baru, serta cara melaporkan keterbatasan model secara terukur.

### 1.5 Ruang Lingkup dan Batasan

Yang dikerjakan:

- Klasifikasi biner teks pesan (phishing/aman) dalam bahasa Indonesia dan Inggris, dengan confidence dan alasan dari lapisan aturan.
- REST API FastAPI, model scikit-learn yang disimpan dengan joblib, container Docker untuk API dan web.
- Aplikasi Flutter untuk Android dan web, riwayat lokal, serta fitur "Scan dengan PhishGuard" di menu seleksi teks dan menu Bagikan Android.

Yang tidak dikerjakan: membaca SMS secara otomatis di latar belakang (membutuhkan izin SMS yang dibatasi Google Play), memblokir atau melaporkan pesan, memeriksa isi tautan atau file APK, menganalisis gambar/screenshot, dan versi iOS.

Sumber data: dataset publik Mishra dan Soni (2022), Rahmi dan Wibisono (2016), bopbi (2021), serta 69 teks pesan penipuan 2022 sampai 2026 yang dikutip apa adanya dari berita dan peringatan resmi (rincian di subbab 2.4).

Keterbatasan model: data latih berupa SMS dan pesan singkat, sehingga hasil untuk email panjang belum diuji. Contoh modus baru masih sedikit. Pada 12 pesan uji modus baru yang ditulis setelah pelatihan, model saja benar 8, dan model dengan lapisan aturan benar 11. Angka 11 tersebut masih optimis karena aturan dirancang setelah melihat pesan yang lolos.

Privasi dan keamanan: isi pesan hanya diproses di memori selama request dan tidak disimpan maupun dicatat di log server. Riwayat disimpan di perangkat pengguna saja. Aplikasi Android hanya meminta izin INTERNET. API membatasi 60 request per menit per alamat IP untuk tiap worker, menolak body di atas 32 KB sebelum JSON dibaca, dan hanya menerima panggilan browser dari origin web aplikasi sendiri. Prototype memakai HTTP di jaringan lokal; deployment publik direncanakan memakai HTTPS.

Asumsi penggunaan: pengguna memiliki koneksi ke server API, dan hasil aplikasi dipakai sebagai bantuan awal. Keputusan untuk mengabaikan atau menindaklanjuti pesan tetap ada pada pengguna.

# BAB II ANALISIS KEBUTUHAN DAN LANDASAN SOLUSI

## 2.1 Profil Pengguna dan Skenario Penggunaan

Aktor utama adalah pengguna ponsel umum yang menerima SMS, pesan WhatsApp, atau email mencurigakan. Mereka membutuhkan jawaban cepat dalam bahasa yang mudah dipahami dan tidak harus memahami istilah teknis. Aktor kedua adalah pengembang/pengelola layanan yang memantau kesehatan API, versi model, dan waktu respons.

User journey utama: pengguna menerima pesan, menyalin teksnya, membuka PhishGuard, menempel teks, menekan Analisis, lalu membaca hasil dan alasan sebelum memutuskan tindakan. Jalur cepat di Android: pengguna menekan lama teks di aplikasi pesan, memilih "Scan dengan PhishGuard" dari menu seleksi teks, dan hasil muncul sebagai popup tanpa membuka aplikasi utama. Di aplikasi SMS dan WhatsApp teks pesan biasanya tidak dapat diseleksi, sehingga pengguna menekan lama pesan, memilih Bagikan, lalu memilih "Scan dengan PhishGuard"; popup yang sama akan muncul.

Skenario normal: teks valid dikirim, API menjawab 200 dengan label, confidence, versi model, latency, dan alasan; aplikasi menampilkan kartu hasil dan menyimpan ringkasan ke riwayat.

Skenario kegagalan:

- Teks kosong atau hanya spasi: aplikasi menampilkan "Teks pesan tidak boleh kosong" tanpa memanggil API; bila tetap terkirim, API menjawab 422.
- Teks lebih dari 2.000 karakter: kolom input membatasi panjang; popup scan memotong teks dan memberi keterangan; API menolak dengan 422.
- Terlalu banyak request dari satu alamat IP dalam semenit: API menjawab 429 dengan header Retry-After, dan aplikasi menampilkan "Terlalu banyak permintaan. Tunggu sebentar lalu coba lagi."
- Server mati atau alamat salah: aplikasi menampilkan "Tidak bisa terhubung ke server" beserta alamat server, tanpa crash. Alamat server dapat diubah di menu pengaturan.

- Prediksi salah: kartu hasil selalu menampilkan saran umum (jangan klik tautan, jangan bagikan OTP/PIN), dan keterbatasan model dijelaskan di subbab 2.6.

## 2.2 Analisis Kebutuhan Fungsional

| ID | Kebutuhan Fungsional | Prioritas | Kriteria Penerimaan |
|---|---|---|---|
| FR-01 | Pengguna memasukkan teks pesan dan sistem menampilkan label (Phishing/Aman) beserta confidence | Must | Teks valid menghasilkan kartu hasil berisi label dan confidence 0 sampai 100% dalam ≤ 3 detik |
| FR-02 | Sistem menolak input kosong dan input lebih dari 2.000 karakter dengan pesan yang jelas | Must | Teks kosong/spasi menampilkan "Teks pesan tidak boleh kosong" tanpa memanggil API; API menjawab 422 untuk kosong dan > 2.000 karakter |
| FR-03 | Sistem menyimpan riwayat analisis di perangkat | Should | Setiap hasil tersimpan (cuplikan 80 karakter, label, confidence, waktu), maksimal 50 entri, dapat dihapus pengguna |
| FR-04 | Sistem menampilkan alasan (tanda bahaya) bila pola penipuan yang dikenal terdeteksi | Should | Pesan berisi file .apk, permintaan kode OTP, kerja dibayar per tugas, atau domain tiruan brand menampilkan alasan yang sesuai |

| | | | |
|---|---|---|---|
| FR-05 | Pengguna Android memeriksa teks dari aplikasi lain lewat menu seleksi teks | Could | "Scan dengan PhishGuard" muncul di menu seleksi teks dan popup hasil tampil walaupun aplikasi belum dibuka sebelumnya |
| FR-06 | Pengguna Android memeriksa SMS atau pesan chat yang teksnya tidak dapat diseleksi lewat menu Bagikan | Could | "Scan dengan PhishGuard" muncul di menu Bagikan untuk teks (ACTION_SEND, text/plain) dan membuka popup hasil yang sama dengan FR-05 |

## 2.3 Kebutuhan Nonfungsional

| ID | Aspek | Target/Kriteria |
|---|---|---|
| NFR-01 | Kinerja/latency | Latency p95 < 3 detik untuk POST /v1/predict (terukur dari sisi klien HTTP: p95 4,5 ms pada 300 request di Docker lokal) |
| NFR-02 | Reliability | Semua request valid dijawab 200 dan input tidak valid dijawab 422 pada uji 300 request; container memiliki health check /health; aplikasi tidak crash saat server mati |
| NFR-03 | Keamanan dan privasi | Isi pesan tidak disimpan dan tidak dicatat di log server (diuji otomatis); riwayat hanya di perangkat; izin Android hanya INTERNET; API hanya memuat berkas model sendiri; POST /v1/predict dibatasi 60 request per menit per IP untuk tiap worker (429), body di atas 32 KB ditolak 413 sebelum JSON dibaca, CORS hanya untuk origin web aplikasi (terukur: 150 request beruntun dari satu IP ke 2 worker menghasilkan 120 jawaban 200 dan 30 jawaban 429; body 40 KB dijawab 413); HTTPS pada deployment publik |

| | | |
|---|---|---|
| NFR-04 | Usability dan aksesibilitas | Tersedia keadaan idle, loading, hasil, dan error; label ditampilkan dengan teks dan ikon, bukan warna saja; pesan error menjelaskan penyebab; layout dibatasi lebar 560 px agar terbaca di layar lebar |
| NFR-05 | Kualitas model | Pada data uji: akurasi ≥ 85% dan F1 kelas phishing ≥ 0,85 (terukur: 97,8% dan 0,921) |

### 2.4 Data, Input, dan Output

Data latih dan uji disusun dari sumber berikut. Pemetaan label mengikuti tujuan aplikasi, yaitu menandai penipuan. Pesan promosi dan spam iklan dimasukkan ke kelas aman karena bukan penipuan.

- Mishra dan Soni (2022), Mendeley Data, CC BY 4.0: 5.971 SMS berbahasa Inggris berlabel ham, spam, smishing. Smishing dipetakan ke phishing; ham dan spam ke aman.
- Rahmi dan Wibisono (2016), CC BY-SA 4.0: 1.143 SMS berbahasa Indonesia berlabel normal, penipuan, promo. Penipuan dipetakan ke phishing; normal dan promo ke aman.
- bopbi (2021), CC0 1.0: 48 SMS Indonesia asli kategori hadiah, pinjaman, promo palsu (phishing) dan SMS premium (aman).
- Kumpulan kutipan 2022 sampai 2026: 69 teks penipuan dan 11 template notifikasi resmi yang dikutip apa adanya dari berita cek fakta dan halaman edukasi bank, masing-masing dengan URL sumber. Sekitar 28 teks penipuan berasal dari unggahan Facebook, bukan SMS.

Preprocessing: normalisasi Unicode NFKC (mengubah huruf dekoratif menjadi huruf biasa), penyatuan spasi, dan penghapusan duplikat. Sebanyak 34 teks yang muncul dengan label berbeda di sumber berbeda dibuang. Hasilnya 7.067 pesan unik: 6.089 aman dan 978 phishing. Data dibagi 80% latih (5.654) dan 20% uji (1.413) secara stratified per label dan sumber dengan seed 42. Lowercasing dan pembentukan fitur dilakukan di dalam pipeline model, sehingga aturan yang sama dipakai saat training dan inference.

Input aplikasi berupa teks pesan (string, 1 sampai 2.000 karakter, tidak boleh hanya spasi). Output berupa label "phishing" atau "aman", confidence 0 sampai 1, versi model, latency server dalam milidetik, dan daftar alasan. Contoh: teks "Akun DANA Anda akan dinonaktifkan dalam 24 jam. Verifikasi segera di dana-verifikasi.id" menghasilkan label phishing, confidence 0,85, dan alasan "Tautan 'dana-verifikasi.id' memakai nama brand tapi bukan domain resmi".

Potensi bias: 82% data uji berbahasa Inggris, dan data Indonesia didominasi modus 2016 (undian berhadiah). Contoh notifikasi resmi bank berbahasa Indonesia sangat sedikit, sehingga model sempat menandai notifikasi transaksi debit BCA asli sebagai phishing. Karena itu evaluasi dilaporkan per bahasa dan per sumber, serta ditambah set uji modus baru yang tidak pernah dipakai untuk pelatihan.

### 2.5 Pemilihan Model, Algoritma, dan Framework

Model. Dua kandidat dibandingkan dengan fitur yang sama, yaitu TF-IDF n-gram kata (1 sampai 2) digabung n-gram karakter (3 sampai 5). Fitur karakter dipakai karena pesan penipuan sering menyamarkan ejaan, misalnya "sh0pee" atau "m-dapatkan". Hasil validasi silang 5-fold pada data latih: Multinomial Naive Bayes mendapat F1 phishing 0,838, sedangkan Logistic Regression dengan class_weight balanced mendapat 0,919. Logistic Regression dipilih. Kedua model diimplementasikan dengan scikit-learn (Pedregosa et al., 2011). Bobot koefisiennya juga dapat dibaca untuk menjelaskan kata yang mendorong suatu prediksi, sesuai pendekatan XAI untuk model linear pada materi Pertemuan 7. Model transformer seperti IndoBERT tidak dipilih untuk iterasi ini karena membutuhkan GPU atau server yang jauh lebih besar, sementara data berlabel Indonesia baru sekitar 1.300 pesan.

Lapisan aturan. Analisis bobot fitur menunjukkan model tidak mengenali ".apk" karena kata itu tidak ada di data latih Indonesia, dan kata "paket" justru condong ke aman karena sering muncul pada SMS paket internet. Karena itu ditambahkan empat aturan regex: lampiran .apk, permintaan mengirim kode

OTP/verifikasi, tawaran kerja dibayar per like/tugas, dan domain yang memakai nama brand tetapi bukan domain resmi. Aturan diuji pada 6.089 pesan aman dan tidak menghasilkan satu pun false alarm.

API framework. FastAPI dipilih dibanding Flask karena validasi request-response dengan Pydantic dan dokumentasi interaktif /docs tersedia langsung, sesuai perbandingan pada materi Pertemuan 2 (FastAPI, n.d.). Validasi panjang dan teks kosong cukup ditulis di skema.

Serialization. Pipeline disimpan dengan joblib (berbasis Pickle) sebagai satu berkas berisi TF-IDF dan classifier, mengikuti materi Pertemuan 3. Pickle dipilih dibanding ONNX karena layanan berjalan di lingkungan Python yang sama dengan pelatihan dan pipeline scikit-learn dapat disimpan utuh tanpa konversi.

Frontend. Flutter dipilih karena satu basis kode dapat dibangun sebagai APK Android dan aplikasi web, dan menjadi frontend utama mata kuliah (Pertemuan 4; Flutter, n.d.). Streamlit dan Gradio cocok untuk demo web cepat tetapi tidak menghasilkan aplikasi Android dan tidak dapat terdaftar di menu seleksi teks Android, yang memakai intent ACTION_PROCESS_TEXT (Google, n.d.).

Container dan deployment. Docker dipakai agar API dan model berjalan dengan versi dependency yang sama di laptop dan di server (Pertemuan 5; Docker, n.d.). Untuk cloud direncanakan Docker Space atau PaaS sesuai Pertemuan 6, karena keduanya menjalankan Dockerfile yang sama dan menyediakan HTTPS.

Pengamanan API. Rate limiting dan batas ukuran body ditulis sebagai dua middleware ASGI kecil di api/app/security.py, bukan memakai pustaka slowapi atau limit_req di nginx. Alasannya, API sudah terlindungi walaupun dijalankan tanpa reverse proxy, dan tidak ada dependency baru yang harus dikunci versinya. Batas body 32 KB dipilih dari ukuran terburuk 2.000 karakter emoji dalam JSON (sekitar 24 KB), sehingga input sah tidak pernah tertolak. Batas 60 request per menit jauh di atas kebutuhan satu pengguna yang memeriksa pesan satu per satu.

Konsekuensinya, penghitung rate limiting tersimpan di memori tiap worker, sehingga dengan 2 worker satu IP dapat mengirim sampai 120 request per menit. Hal ini terukur pada pengujian dan diterima untuk skala prototype.

### 2.6 Risiko dan Etika AI

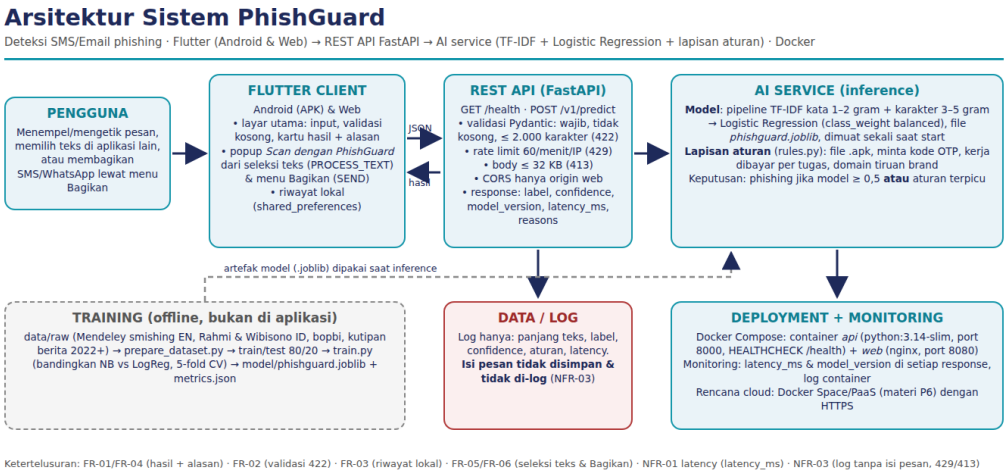
- Pesan penipuan lolos (false negative): pengguna merasa aman lalu tertipu. Mitigasi: recall phishing dipantau sebagai metrik utama bersama F1, lapisan aturan untuk modus yang dikenal, saran keamanan umum tetap ditampilkan pada hasil aman, dan teks antarmuka menyebut hasil sebagai bantuan awal.
- Pesan asli ditandai phishing (false positive): pengguna mengabaikan notifikasi penting. Mitigasi: aturan diuji agar tidak memicu pada pesan aman, alasan ditampilkan agar pengguna dapat menilai sendiri, dan contoh notifikasi resmi ditambah ke data latih.
- Bias data: dominasi bahasa Inggris dan modus lama. Mitigasi: evaluasi per bahasa, per sumber, dan set uji modus baru; anggota kelompok mengumpulkan SMS asli dengan nama dan nomor disamarkan.
- Privasi: pesan dapat memuat data pribadi. Mitigasi: isi pesan tidak disimpan atau dicatat di log server (diuji otomatis), riwayat hanya di perangkat, dan izin aplikasi Android hanya INTERNET.
- Keamanan: berkas Pickle dapat menjalankan kode bila berasal dari sumber tidak tepercaya. Mitigasi: API hanya memuat berkas model yang dibuat oleh skrip pelatihan di repository sendiri. Koneksi HTTP lokal diganti HTTPS saat deployment publik.
- Penyalahgunaan API publik: request dalam jumlah besar dapat membebani server, dan situs lain dapat memanggil API dari browser pengunjungnya. Mitigasi yang sudah diterapkan dan diuji: rate limiting 60 request per menit per IP untuk tiap worker (429), body di atas 32 KB ditolak 413 sebelum JSON dibaca, CORS hanya untuk origin web aplikasi, serta batas 64 koneksi dan keep-alive 5 detik pada server.
- API sebagai oracle: karena response memuat confidence dan alasan, penyerang dapat mengubah pesan penipuan sedikit demi sedikit dan

mengirimnya berulang sampai label berubah menjadi aman. Risiko ini melekat pada fitur penjelasan yang justru dibutuhkan pengguna, sehingga alasan tetap ditampilkan. Rate limiting memperlambat percobaan berulang tetapi tidak menghilangkan risiko ini, sehingga dicatat sebagai keterbatasan dan dibahas lagi di laporan akhir.

- Ketergantungan layanan: aplikasi tidak dapat memberi hasil bila server mati. Mitigasi: health check pada container, pesan kegagalan yang jelas di aplikasi, dan pengaturan alamat server.
- Perubahan modus (data drift): pola penipuan baru membuat kualitas prediksi turun setelah deployment. Mitigasi: versi model dicantumkan pada setiap response, dan model dilatih ulang serta diuji dengan set uji baru ketika data bertambah.

# BAB III RANCANGAN ARSITEKTUR DAN API

## 3.1 Diagram Arsitektur Sistem




*Gambar 3.1 Arsitektur sistem PhishGuard*

Pengguna berinteraksi dengan klien Flutter (APK Android atau web). Klien mengirim teks ke REST API FastAPI dalam format JSON. API memvalidasi input, memanggil AI service yang menjalankan model dan lapisan aturan, lalu mengembalikan hasil. Log server hanya mencatat panjang teks, label, confidence, aturan yang terpicu, dan latency. API dan web dijalankan sebagai dua container Docker dengan health check. Pelatihan model berjalan terpisah (offline) dan menghasilkan berkas phishguard.joblib yang dimuat API saat start. Monitoring di cloud dan HTTPS masih berupa rencana (subbab 4.4).

## 3.2 Alur Sistem dan Inference

- Input: pengguna menempel teks di layar utama atau memilih teks lalu menekan "Scan dengan PhishGuard", atau membagikan pesan lewat menu Bagikan.
- Validasi di klien: teks kosong ditolak dengan pesan; panjang dibatasi 2.000 karakter.

- Request: klien mengirim POST /v1/predict dengan body JSON {"text": ...} ke alamat server yang tersimpan (default 10.0.2.2:8000 untuk emulator, localhost:8000 untuk web).
- Pengamanan sebelum validasi: request dari IP yang sudah melewati 60 request dalam semenit dijawab 429, dan body di atas 32 KB dijawab 413 sebelum JSON dibaca.
- Validasi di server: Pydantic memeriksa field text wajib ada, tidak hanya spasi, dan maksimal 2.000 karakter. Bila gagal, API menjawab 422 tanpa menjalankan model.
- Preprocessing dan inference: pipeline mengubah teks menjadi fitur TF-IDF kata dan karakter, lalu Logistic Regression menghitung peluang phishing.
- Postprocessing: lapisan aturan memeriksa teks. Label phishing diberikan bila peluang phishing minimal 0,5 atau ada aturan yang terpicu. Confidence adalah nilai terbesar antara peluang model dan skor aturan (0,85 sampai 0,95) untuk hasil phishing, atau peluang aman untuk hasil aman.
- Response: API menjawab 200 dengan label, confidence, model_version, latency_ms, dan reasons, lalu mencatat log tanpa isi pesan.
- Visualisasi: klien menampilkan kartu hasil berwarna dengan ikon dan teks label, bar confidence, daftar tanda bahaya, saran tindakan, dan menyimpan ringkasan ke riwayat lokal.

Alur kegagalan: bila koneksi gagal atau melewati batas waktu 10 detik, klien menampilkan pesan bahwa server tidak dapat dihubungi; status 422 ditampilkan sebagai pesan input tidak valid; status 429 ditampilkan sebagai permintaan terlalu banyak; status 413 ditampilkan sebagai pesan terlalu besar; status lain ditampilkan sebagai kesalahan server dengan kodenya.

### 3.3 Rancangan Endpoint API

| Method | Endpoint | Request | Response | Status/Error |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| POST | /v1/predict | {text: string, wajib, 1 sampai 2.000 karakter, tidak hanya spasi} | {label, confidence, model_version, latency_ms, reasons[]} | 200 berhasil; 413 body di atas 32 KB; 422 validasi gagal; 429 lebih dari 60 request per menit per IP (header Retry-After); 500 kesalahan server |
| GET | /health | - | {status: "ok", model_version} | 200; bila model gagal dimuat server tidak start dan health check container gagal |
| GET | /docs | - | Dokumentasi interaktif OpenAPI (dibuat otomatis FastAPI) | 200 |

### 3.4 Struktur Data Request-Response

Request POST /v1/predict (Content-Type: application/json):

```
{
  "text": "Ini foto paket kakak ya, tolong dicek dulu:
LIHAT_FOTO_PAKET.apk"
}
```

Aturan validasi: text bertipe string, wajib, maksimal 2.000 karakter, dan tidak boleh kosong atau hanya spasi.

Response 200:

```
{
  "label": "phishing",
  "confidence": 0.95,
  "model_version": "tfidf_logreg-20261111",
  "latency_ms": 4.3,
  "reasons": [
    "Meminta membuka/menginstal file .apk (modus APK
undangan/paket/tilang)"
  ]
}
```

Tipe data: label string ("phishing" atau "aman"), confidence float 0 sampai 1, model_version string (nama model dan tanggal pelatihan), latency_ms float

(waktu inference di server), reasons array string yang kosong bila tidak ada aturan yang terpicu.

Response 422 (contoh teks hanya spasi), mengikuti format validasi FastAPI:

```
{
  "detail": [ {
    "type": "value_error",
    "loc": ["body", "text"],
    "msg": "Value error, Teks pesan tidak boleh kosong"
  } ]
}
```

GET /health menjawab {"status": "ok", "model_version": "tfidf_logreg-20261111"}.

### 3.5 Rancangan Serialization Model

Format: joblib (berbasis Pickle). Berkas model/phishguard.joblib berisi dictionary {pipeline, version, name}. Pipeline memuat FeatureUnion TF-IDF kata dan karakter beserta classifier Logistic Regression, sehingga praproses saat inference sama dengan saat training.

Versi model: nama model dan tanggal pelatihan (contoh tfidf_logreg-20261111), dikirim di setiap response dan di /health. Metrik setiap pelatihan disimpan di model/metrics.json, termasuk hasil pemeriksaan NFR kualitas model.

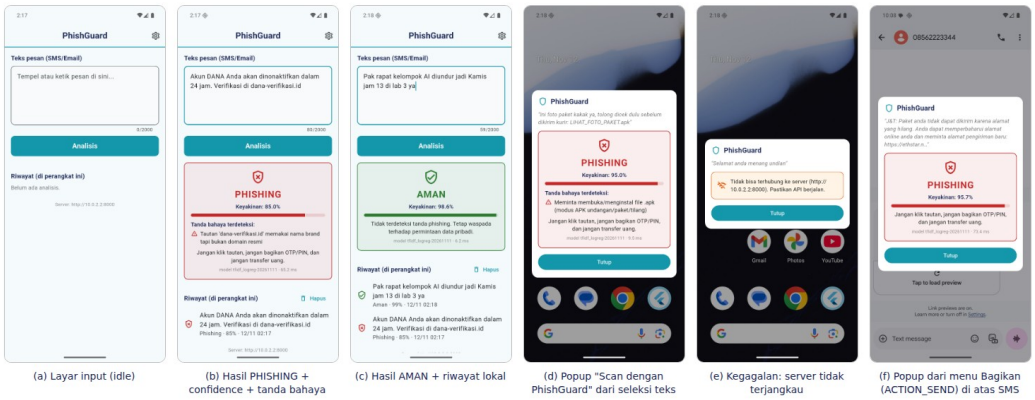
Dependency: Python 3.14, scikit-learn 1.9.1, joblib 1.6.0, numpy 2.5.4, scipy 1.18.1. Versi ini dikunci di api/requirements.txt karena berkas Pickle harus dimuat dengan versi library yang sama dengan saat pelatihan.

Proses loading: berkas dimuat satu kali saat server start (lokasi dapat diatur lewat variabel MODEL_PATH), bukan pada setiap request.

Fallback: bila berkas gagal dimuat, server tidak start dan health check container gagal, sehingga masalah terlihat sebelum pengguna mengirim request. Pemulihan dilakukan dengan menjalankan kembali image Docker versi sebelumnya. ONNX dicatat sebagai alternatif bila model perlu dijalankan di luar Python.

### 3.6 Rancangan Antarmuka Pengguna

Antarmuka mengikuti mockup Checkpoint 1 dan sudah diimplementasikan. Gambar 3.2 adalah screenshot aplikasi Android di emulator. Saat menunggu response, tombol Analisis dinonaktifkan dan menampilkan indikator loading. Label hasil ditulis sebagai teks dan ikon, sehingga warna bukan satu-satunya penanda.




*Gambar 3.2 Antarmuka PhishGuard di Android: (a) input, (b) hasil phishing dengan tanda bahaya, (c) hasil aman dan riwayat, (d) popup scan dari seleksi teks, (e) server tidak terjangkau, (f) popup dari menu Bagikan di atas percakapan SMS*

# BAB IV RENCANA IMPLEMENTASI DAN PENGUJIAN

## 4.1 Strategi Implementasi

Implementasi dilakukan bertahap dan setiap tahap diuji sebelum tahap berikutnya:

- Data dan model: skrip prepare_dataset.py membangun data latih/uji dari data mentah; train.py membandingkan Naive Bayes dan Logistic Regression, menyimpan model terbaik dan metrik.
- Backend API: FastAPI dengan skema Pydantic, pemuatan model sekali saat start, lapisan aturan, CORS yang dibatasi ke origin web, rate limiting dan batas ukuran body sebagai middleware ASGI, dan pytest.
- Frontend Flutter: layar utama, kartu hasil, riwayat lokal, pengaturan alamat server; kemudian popup Scan dengan PhishGuard melalui Activity Android yang menerima ACTION_PROCESS_TEXT, lalu menu Bagikan (ACTION_SEND teks) pada Activity yang sama untuk aplikasi SMS dan chat.
- Container: Dockerfile API (python:3.14-slim, pengguna non-root, health check) dan Dockerfile web (nginx), dijalankan bersama dengan docker-compose.yml.
- Deployment cloud dan monitoring: dikerjakan sesuai jadwal Pertemuan 6 dan 7 (subbab 4.4).

Tahap 1 sampai 4 sudah berjalan sebagai prototype dan lulus pengujian pada 12 November 2026. Tahap 5 belum dikerjakan.

## 4.2 Struktur Repository

```
PhishGuard/
├── api/
│   ├── app/main.py        endpoint FastAPI
│   ├── app/rules.py       lapisan aturan
│   ├── tests/             pytest
│   ├── Dockerfile
│   └── requirements.txt
├── model/                 phishguard.joblib, metrics.json
├── src/
│   ├── data/prepare_dataset.py
│   └── model/train.py
```

```
├── data/
│   ├── raw/                data asli dari sumber
│   ├── processed/          train.csv, test.csv
│   ├── challenge/          uji modus baru
│   └── README.md           lisensi dan sitasi
├── mobile/                 aplikasi Flutter
│   ├── lib/                main, scan, api, history, widgets
│   ├── test/               widget test
│   ├── android/            ScanTextActivity, manifest
│   └── Dockerfile.web
├── docs/                   checkpoint, proposal, laporan
├── plans/                  rencana dan laporan pengujian
├── mockups/                diagram dan screenshot
├── docker-compose.yml
├── requirements-dev.txt
└── README.md
```

Kode, data, model, dan dokumentasi disimpan di repository GitHub publik https://github.com/lildwagz/phishguard. Kontribusi setiap anggota dicatat melalui commit dan issue di repository tersebut, sesuai materi Pertemuan 11.

### 4.3 Rencana Pengujian

Pengujian mencakup unit test (36 tes pytest untuk API, aturan, dan pengamanan; 9 widget test Flutter), integration test API di container, end-to-end di browser dan emulator Android, pengukuran latency, dan evaluasi model. Hasil pengujian 12 November 2026 tercatat di plans/2026-11-12-full-testing.md. Uji beban dengan request bersamaan (concurrent) belum dilakukan dan dijadwalkan pada Pertemuan 13; batas RATE_LIMIT_PER_MIN dinaikkan sementara saat uji beban agar hasilnya tidak didominasi 429. Menu Bagikan (FR-06) sudah diuji di HP Android asli pada 12 November 2026: SMS yang dibagikan ke PhishGuard memunculkan popup hasil.

Evaluasi model memakai akurasi, precision, recall, dan F1 kelas phishing pada 1.413 pesan uji, dilaporkan juga per bahasa. Hasil saat ini: akurasi 0,978, precision 0,923, recall 0,918, F1 0,921 (Inggris 0,901, Indonesia 0,944). Dari 196 pesan phishing uji, 16 lolos; dari 1.217 pesan aman, 15 salah ditandai. Sebagai pembanding, menebak semua pesan aman hanya menghasilkan akurasi 0,861 dan F1 0.

| ID | Jenis Uji | Skenario | Expected Result | Bukti |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| T-01 | Integration | POST /v1/predict dengan teks valid ke container API | 200, body sesuai skema (label, confidence, model_version, latency_ms, reasons) | pytest + uji live (full-testing.md baris 4) |
| T-02 | Negative test | Teks kosong, hanya spasi, 2.001 karakter, field hilang | 422 dengan pesan "Teks pesan tidak boleh kosong" atau batas panjang; teks 2.000 karakter diterima | pytest + uji live |
| T-03 | Performance | 300 request berurutan ke Docker lokal; uji concurrent pada Pertemuan 13 | p95 < 3 detik, tanpa request gagal (terukur p95 4,5 ms, 0 gagal) | full-testing.md baris 4c |
| T-04 | Unit (aturan) | 19 kasus positif/negatif, termasuk OTP resmi "jangan bagikan kode" | Aturan terpicu hanya pada pola penipuan; 0 false alarm pada 6.089 pesan aman | api/tests/ test_rules.py |
| T-05 | Privasi | Kirim teks unik lalu periksa log | Teks tidak muncul di log, hanya panjang/label/latency | test_api.py + grep log container |

| | | | | |
|---|---|---|---|---|
| T-06 | End-to-end | Web (Chrome) dan APK di emulator Android 36: kosong, phishing, aman, scan seleksi teks, server dimatikan; APK di HP Android asli: SMS dibagikan lewat menu Bagikan | Hasil dan pesan error tampil sesuai; popup scan berjalan dari cold start; popup muncul dari menu Bagikan | Screenshot Gambar 3.2 (d) dan (f); uji manual di HP |
| T-07 | Evaluasi model | 1.413 pesan uji dan 21 pesan modus baru yang tidak dipakai pelatihan | Akurasi ≥ 85% dan F1 ≥ 0,85 (terukur 97,8% dan 0,921) | model/ metrics.json |
| T-08 | Keamanan | 150 request beruntun dari satu IP; body 40 KB biasa dan chunked; preflight CORS dari origin lain | 429 setelah batas dengan Retry-After; 413; origin lain tanpa header Access-Control-Allow-Origin (terukur: 120 x 200 lalu 30 x 429, 413, 400) | api/tests/ test_security.py + uji live Docker |

### 4.4 Rencana Deployment dan Monitoring

Environment: Docker Compose menjalankan container api (port 8000) dan web (port 8080). Image API memakai python:3.14-slim, menyalin hanya kode API, requirements, dan berkas model (diatur .dockerignore), berjalan sebagai pengguna non-root, dan memiliki HEALTHCHECK ke /health setiap 30 detik.

Platform cloud: direncanakan Docker Space atau PaaS sesuai materi Pertemuan 6. Dockerfile perlu disesuaikan dengan port yang ditetapkan platform (misalnya 7860 pada Docker Space), lalu aplikasi Flutter diarahkan ke URL HTTPS dan pengaturan HTTP lokal di Android dihapus. Platform final dipilih kelompok bersama dosen.

Secrets: layanan saat ini tidak memakai kredensial. Konfigurasi publik seperti MODEL_PATH, port, CORS_ORIGINS, RATE_LIMIT_PER_MIN, dan MAX_BODY_BYTES diatur lewat environment di docker-compose.yml dan dipisahkan dari secret. Bila API dipasang di belakang reverse proxy, FORWARDED_ALLOW_IPS diisi alamat proxy agar rate limiting memakai IP klien asli, dan secret tidak boleh di-commit ke repository.

Logging dan metrics: setiap request mencatat panjang teks, label, confidence, aturan yang terpicu, dan latency, tanpa isi pesan. Latency inference di server (latency_ms) dan versi model dikirim di setiap response. Error rate dihitung dari proporsi status selain 200 pada log. Pengukuran waktu respons HTTP saat ini: p50 3,9 ms, p95 4,5 ms, maksimum 6,5 ms pada 300 request berurutan di Docker lokal, tanpa request gagal.

Versioning dan rollback: setiap model memiliki versi di model_version, dan setiap versi stabil diberi tag image Docker serta commit. Bila versi baru gagal health check atau metriknya turun, layanan dikembalikan ke image dan commit stabil sebelumnya.

# BAB V MANAJEMEN PROJECT

## 5.1 Pembagian Peran dan Tanggung Jawab

| Nama/NIM | Peran | Tanggung Jawab | Bukti Kontribusi |
|---|---|---|---|
| [Ketua] | Project lead/backend | Koordinasi, REST API FastAPI, lapisan aturan, integrasi | [Commit/log] |
| [Anggota 1] | AI/model | Pengumpulan dan pelabelan data, pelatihan dan evaluasi model | [Commit/log] |
| [Anggota 2] | Frontend/UI | Aplikasi Flutter (Android dan web), popup scan, riwayat | [Commit/log] |
| [Anggota 3] | Testing/ deployment | Pengujian, Docker, deployment cloud, monitoring | [Commit/log] |

## 5.2 Jadwal Pelaksanaan

| Minggu | Target | PIC | Luaran/Bukti |
|---|---|---|---|
| 1 | Analisis kebutuhan dan studi kasus (Checkpoint 1) | [Nama] | Checkpoint1_PhishGuard_ RancanganAwal.docx |
| 2 | Rancangan API dan skema input-output (FastAPI) | [Nama] | Spesifikasi endpoint, skema Pydantic |
| 3 | Model inference dan serialization | [Nama] | model/phishguard.joblib, metrics.json |
| 4 | Frontend Flutter; pengumpulan Tugas 1 (proposal) | [Nama] | Proposal, diagram, slide presentasi |

| | | | |
|---|---|---|---|
| 5 | Containerization Docker | [Nama] | Dockerfile, docker-compose.yml |
| 6 | Deployment cloud (Docker Space/PaaS) | [Nama] | URL HTTPS layanan |
| 7 | Monitoring, latency, XAI | [Nama] | Log, metrik latency, tampilan alasan |
| 8 | UTS: demo prototype (live app) | [Nama] | Demo |
| 9 | Integration testing skenario input-output | [Nama] | Test report |
| 10 | Penyempurnaan UI/UX | [Nama] | Build APK/web terbaru |
| 11 | Progress report 1: demo arsitektur API | [Nama] | Demo, slide |
| 12 | Progress report 2: demo UI dan alur sistem | [Nama] | Demo, slide |
| 13 | Stress test dan optimasi latency | [Nama] | Laporan uji beban |
| 14 | Dokumentasi sistem dan finalisasi | [Nama] | README, dokumentasi API, laporan |
| 15 | Demo end-to-end | [Nama] | Demo |
| 16 | UAS: presentasi dan penyerahan project | [Nama] | Laporan akhir, repository, URL live |

### 5.3 Risiko Project dan Mitigasi

| Risiko | Dampak | Probabilitas | Mitigasi | PIC |
|---|---|---|---|---|
| Data penipuan Indonesia modus baru sedikit, model melewatkan pesan penipuan | T | T | Lapisan aturan; anggota mengumpulkan SMS asli; latih ulang dan uji dengan set modus baru | [Nama] |
| Notifikasi resmi ditandai phishing (false positive) | S | S | Tambah contoh notifikasi resmi ke data latih; tampilkan alasan agar pengguna dapat menilai | [Nama] |
| Laptop kelompok kehabisan RAM saat build APK sambil menjalankan emulator (pernah terjadi) | S | S | Build APK sebelum emulator dinyalakan; uji di HP Android asli | [Nama] |
| Versi scikit-learn berbeda membuat berkas model gagal dimuat | T | R | Versi dependency dikunci di requirements.txt; latih ulang bila versi diubah | [Nama] |
| Deployment cloud tertunda atau platform berubah | S | S | Dockerfile yang sama untuk lokal dan cloud; demo lokal dengan Docker Compose sebagai cadangan | [Nama] |

| | | | | |
|---|---|---|---|---|
| API publik dibanjiri request atau dipakai berulang untuk menguji pesan penipuan sampai lolos (oracle) | S | S | Rate limiting 60/menit per IP, body maks 32 KB, CORS dibatasi; risiko oracle diterima sebagai trade-off fitur alasan dan dicatat sebagai keterbatasan | [Nama] |

# DAFTAR PUSTAKA

Al Hakim, T. (2026, 9 Januari). OJK terima 411.055 laporan penipuan transaksi keuangan, kerugian capai Rp 9 triliun. Investortrust.id. https://investortrust.id/financial/90898/ojk-terima-411055-laporan-penipuan-transaksi-keuangan-kerugian-capai-rp-9-triliun

Bank Central Asia. (2025, 26 Januari). Hati-hati penipuan smishing mengatasnamakan BCA. https://www.bca.co.id/id/informasi/awas-modus/2025/01/26/15/08/Hati-Hati-Penipuan-Smishing-Mangatasnamakan-BCA

bopbi. (2021). Indonesia SMS spam dataset [Data set]. GitHub. https://github.com/bopbi/indonesia-sms-spam-dataset

Docker. (n.d.). Docker documentation. Diakses 12 November 2026, dari https://docs.docker.com

FastAPI. (n.d.). FastAPI documentation. Diakses 12 November 2026, dari https://fastapi.tiangolo.com

Flutter. (n.d.). Flutter documentation. Diakses 12 November 2026, dari https://docs.flutter.dev

Google. (n.d.). Intent.ACTION_PROCESS_TEXT. Android Developers. Diakses 12 November 2026, dari https://developer.android.com/reference/android/content/Intent#ACTION_PROCESS_TEXT

Mishra, S., & Soni, D. (2022). SMS phishing dataset for machine learning and pattern recognition (Version 1) [Data set]. Mendeley Data. https://doi.org/10.17632/f45bkkt8pr.1

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É.

(2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Rahmi, F., & Wibisono, Y. (2016). Aplikasi SMS spam filtering pada Android menggunakan naive Bayes [Naskah tidak dipublikasikan]. Dataset v1.0 (2018). https://yudiwbs.wordpress.com/2018/08/05/dataset-klasifikasi-bahasa-indonesia-sms-spam-klasifikasi-teks-dengan-scikit-learn/

Tim Cek Fakta. (2022, 9 Desember). Awas, beredar pesan WA berisi file APK mengatasnamakan tagihan PLN. Kompas.com. https://www.kompas.com/cekfakta/read/2022/12/09/144037282/awas-beredar-pesan-wa-berisi-file-apk-mengatasnamakan-tagihan-pln

Tim Cek Fakta. (2023, 4 Agustus). [HOAKS] Tawaran kerja paruh waktu terkait promosi di TikTok. Kompas.com. https://www.kompas.com/cekfakta/read/2023/08/04/140000382/-hoaks-tawaran-kerja-paruh-waktu-terkait-promosi-di-tiktok

## LAMPIRAN

Lampiran 1. Diagram arsitektur resolusi penuh: mockups/arsitektur_v2.png (sumber HTML: mockups/arsitektur_v2.html).

Lampiran 2. Screenshot antarmuka: mockups/screenshots/ (Android: idle, hasil phishing, hasil aman, popup scan, server mati; web: validasi input kosong).

Lampiran 3. Contoh request-response lengkap: subbab 3.4 dan dokumentasi interaktif API di /docs saat server berjalan.

Lampiran 4. Laporan pengujian: plans/2026-11-12-full-testing.md. Metrik model: model/metrics.json.

Lampiran 5. Sumber, lisensi, dan pemetaan label dataset: data/README.md.

Lampiran 6. Bukti pembagian tugas: [diisi kelompok, misalnya log commit per anggota].

## CHECKLIST KELENGKAPAN PROPOSAL

| Status | Komponen yang Diperiksa |
|---|---|
| [x] | Analisis kebutuhan dan persoalan computing jelas. |
| [x] | Diagram arsitektur, API, dan skema input-output lengkap. |
| [x] | Pemilihan model, metode, framework, dan platform memiliki argumentasi. |
| [x] | Rencana implementasi, testing, deployment, monitoring, dan pembagian peran tersedia. |
| [x] | Sitasi, daftar pustaka, dan penggunaan data telah diperiksa. |